CSE 4345: Big Data Analytics

Week 8, Part 2 — Seminal Research & Case Study

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Part 2 Overview — Week 8

Today's Agenda

1. Research Paper — Kreps, Narkhede & Rao (2011): Kafka
2. Case Study — Netflix Data Pipeline (2015–2021)
3. Live Exercise

Seminal Paper Covered

1. Kreps, J., Narkhede, N., & Rao, J. (2011). *Kafka: A Distributed Messaging System for Log Processing.* NetDB Workshop at VLDB 2011. Seattle, WA.

Case Study

Netflix Data Pipeline (2015–2021): Kafka-backed Lambda/Kappa architecture serving **200M subscribers**, processing **100 billion events per day** for real-time personalisation and A/B testing.

CLO Addressed

CLO3, CLO4 — Explain distributed messaging;

Research Paper — Kreps, Narkhede & Rao (2011): Kafka

Research Landscape: Distributed Messaging Timeline

Why Existing Systems Failed at LinkedIn's Scale

By 2010 LinkedIn was generating **billions of activity events per day** (profile views, connection clicks, job searches). ActiveMQ and RabbitMQ were designed for *transactional messaging* (few thousand messages/sec, guaranteed delivery, complex routing). LinkedIn needed **sustained 100 MB/s throughput per broker**, multi-day retention, and replay — none of which traditional MOM systems provided.

Kreps et al. (2011): Publication Context

Full Citation

Kreps, J., Narkhede, N., & Rao, J. (2011). **Kafka: A Distributed Messaging System for Log Processing.** In *Proceedings of the NetDB Workshop at the 37th International Conference on Very Large Data Bases (VLDB)*. Seattle, WA, USA.

Venue: NetDB Workshop at VLDB — focused workshop on networked and data-centre databases
Cited by: >3,500 (Google Scholar, 2024)
Authors: Jay Kreps (principal engineer), Neha Narkhede, Jun Rao — LinkedIn Data Infrastructure team

The Core Abstraction

"Kafka is a distributed, partitioned, replicated commit log service."

Every message is appended to an **ordered, immutable log** per topic-partition. Consumers read by advancing an **offset** — a simple integer pointer into the log. The broker never tracks what a consumer has read; the consumer does. Replaying a stream means resetting the offset to any past position.

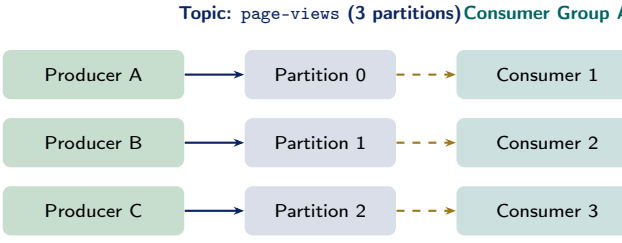
The LinkedIn Origin Story

LinkedIn needed to collect activity data for its **People**

Technical Contribution 1 — Topics, Partitions, Consumer Groups

The Kafka Data Model

- **Topic**: a named, ordered log — analogous to a database table but append-only
- **Partition**: a topic is split into P partitions; each partition is an independent ordered log on one broker. Partitions enable **parallelism** and **horizontal scale**
- **Offset**: each message within a partition has a unique monotonically increasing integer offset. The consumer tracks its *own* offset (stored in `__consumer_offsets` topic, Kafka 0.9+)
- **Consumer Group**: a set of consumers that together consume all partitions of a topic. Each partition is consumed by **exactly one** consumer in the group at a

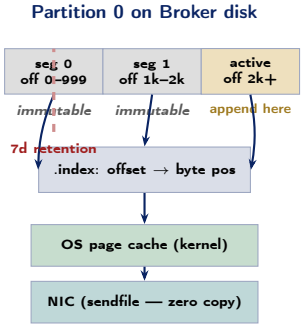


Technical Contribution 2 — Log Storage & Zero-Copy

Partition as a Segmented Append-Only Log

Each partition is stored as a sequence of **segment files** on the broker's local disk:

- Each segment: one `.log` file (message data) + one `.index` file (offset → byte position)
- Active segment: the last segment, open for appends
- Old segments: closed, immutable; eligible for deletion after the **retention period** (default 7 days) or when total log size exceeds a threshold
- **Sequential writes only**: the OS page cache prefetches the next pages, achieving near-disk-bandwidth throughput without SSD



Zero-Copy Transfer (sendfile)

Technical Contribution 3 — Performance Benchmarks

Producer Throughput (from the paper)

Benchmark: single broker, 6× 7200 RPM SATA, 10 GbE NIC, 100-byte messages, no replication.

System	MB/s	Msgs/sec
ActiveMQ	4.3	43,000
RabbitMQ	14.7	147,000
Kafka	**50.5**	**505,000**

Consumer Throughput

System	MB/s	Msgs/sec
ActiveMQ	4.0	40,000

Why Kafka is Faster: Summary

1. **Sequential disk writes**: spinning disk sequential I/O (∼600 MB/s) vs. random I/O (∼100 MB/s) — 6× advantage
2. **Zero-copy**: `sendfile()` eliminates 2 user-space copies and 2 context switches
3. **Batch compression**: producer batches messages into a single compressed record set before sending; compression ratio 5–10× for log data
4. **No per-message ACK**: broker ACKs a batch, not each message individually
5. **Consumer pull**: broker never maintains per-consumer queues; a single log serves all consumers

Impact Today — Kreps et al. (2011)

Kafka's Lasting Influence

- **Kafka Streams** (2016): lightweight stream processing library within Kafka — stateful transformations without Flink or Spark
- **ksqlDB** (2017): streaming SQL on Kafka topics; `CREATE STREAM AS SELECT` pushes continuous query results into a new topic
- **Kafka Connect** (2015): plug-in connectors for 200+ source/sink systems (JDBC, S3, Elasticsearch); replaces Sqoop and Flume for most use cases
- **Schema Registry** (Confluent, 2015): Avro/Protobuf schema enforcement per topic; consumer can deserialise messages written by any past producer version
- **Exactly-once semantics** (Kafka 0.11, 2017)

Harvard CS265 / CMU 15-721 Perspective

- Harvard CS265 presents Kafka's partition-offset model as an instance of **total order broadcast** — equivalent to Paxos log replication but implemented with leader-based ISR (In-Sync Replicas)
- CMU 15-721 uses Kafka as the canonical example of a **log-structured storage engine**: the partition log is exactly a single-writer LSM-tree without compaction — because messages are never updated
- Both courses note that Kafka's **consumer group rebalancing** (partition reassignment on join/leave) is a distributed systems challenge equivalent to consistent hashing

Case Study — Netflix Data Pipeline (2015–2021)

Netflix's Event-Driven Scale

Why Netflix Needed an Industrial-Scale Pipeline

Netflix (200M subscribers, 2021) generates:

- **Playback events**: play, pause, seek, buffering, quality switch — for every stream on every device
- **UI interactions**: hover, click, scroll on the home screen catalogue — drives A/B testing
- **Search queries**: what users search, what they select from results
- **Error events**: client-side errors, CDN cache misses, codec failures — for reliability monitoring
- **Scale**: $\approx$**100 billion events/day**, peaking at **1.3 million events/second** during prime-time (Fri 9 pm US Eastern)

All events flow through **Kafka** before being processed

What Fails Without This Pipeline

- **Personalisation lag**: if the home screen re-ranks titles only once per day, a user who just finished *Stranger Things S4* still sees S3 at the top the next morning
- **A/B testing blindness**: Netflix runs >200 concurrent A/B tests; without real-time event ingestion, test results take days instead of hours
- **Reliability gaps**: stream rebuffering events alert the CDN team; without real-time stream processing, a region-wide outage takes 30 minutes to detect

Netflix Lambda + Kappa Architecture

Lambda Architecture at Netflix

Speed layer (latency <1 minute):

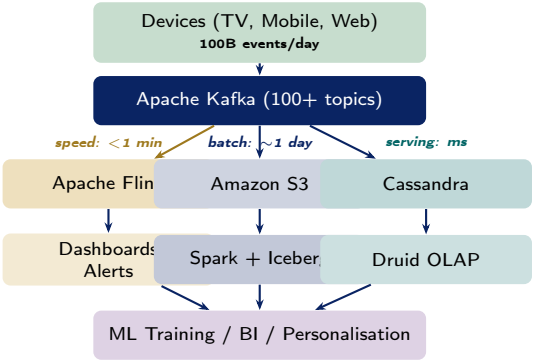
- Kafka → **Flink** (event-time windowing)
- Outputs: real-time dashboards (Kibana), alert triggers, live A/B test counters

Batch layer (latency ∼1 day):

- Kafka → S3 (via Kafka S3 sink connector)
- S3 → **Spark + Apache Iceberg** (hourly/daily ETL jobs on EMR)
- Outputs: ML training datasets, billing aggregates, long-term analytics warehouse

Serving layer:

- **Cassandra**: low-latency user-state reads (last watched, continue-watching row)



Netflix A/B Testing Pipeline on Kafka

Why A/B Testing Requires Real-Time Events

Netflix runs >**200 simultaneous A/B tests**: different artwork thumbnails, autoplay delays, recommendation algorithms, UI layouts. Each test needs:

1. **Assignment**: user bucketed into control or treatment at login time (stored in Cassandra)
2. **Exposure event**: logged to Kafka when the user sees the tested feature
3. **Outcome events**: play, click, skip, subscribe — logged to Kafka within seconds of the action
4. **Statistical aggregation**: Flink window joins assignment $\bowtie$ exposure $\bowtie$ outcome events to compute conversion rates *in real time*
5. **Result**: experiment dashboard updated every **5**

Scale Numbers (Netflix, 2021)

Metric	Value
Subscribers	200M
Events per day	100 billion
Peak events per second	1.3 million
Kafka topics	4,000+
Kafka brokers	2,000+
Message retention	7 days
Flink jobs (speed layer)	150+
Concurrent A/B tests	>200
Experiment result latency	5 minutes
S3 batch pipeline cadence	Hourly
Iceberg table count	5,000+

Lessons Learned & Discussion Questions

Key Lessons from Netflix's Pipeline

1. **Kafka as the single source of truth**: all events written to Kafka first; the speed and batch layers are both consumers of the same topic — eliminating the data inconsistency problem of classic Lambda (two separate ingestion paths)
2. **Retention enables reprocessing**: 7-day retention lets Netflix replay any event window when a Flink job has a bug or a Spark job fails
3. **Kappa simplification**: Netflix increasingly moved batch jobs to Flink stream processing (processing historical S3 data as a bounded stream), reducing pipeline complexity
4. **Schema Registry is non-negotiable at scale**: 4,000 topics $\times$ evolving schemas $\Rightarrow$ consumer breakage without schema governance

Discussion Questions (CLO3, CLO4)

1. Netflix's Kafka cluster handles 1.3M events/sec at peak across 4,000 topics. If each topic has 12 partitions, calculate the **minimum number of brokers** needed assuming each broker handles 50,000 partitions and a replication factor of 3. *[CLO4 — Apply]*
2. A Flink job joins three Kafka streams: `assignments`, `exposures`, and `outcomes` on `user_id`. Events may arrive out of order by up to 30 seconds. Design an **event-time window** strategy (window size, watermark lag, allowed lateness) and justify each choice. *[CLO4 — Design]*
3. Netflix uses both Lambda (speed +

Live Exercise

Live Exercise — Kafka Producer/Consumer in Python

Task (25 minutes, pairs)

Simulate a Kafka producer/consumer pipeline using the **kafka-python** library with a local single-broker cluster (Docker Compose provided below).

Goals:

1. Run the producer — send 1,000 synthetic Netflix play events to topic `play-events`.
2. Run the consumer — read all messages, count plays per `content_id`.
3. Reset the consumer offset to the *beginning* and re-consume — verify idempotent results.
4. Change the consumer group ID and observe that a new group starts from offset 0.

Quick-start

Python Skeleton

```
import json, random, time
from kafka import KafkaProducer, KafkaConsumer

TOPIC   = "play-events"
BROKER  = "localhost:9092"
CONTENT = [f"show_{i}" for i in range(20)]

# --- Producer ---
producer = KafkaProducer(
  bootstrap_servers=BROKER,
  value_serializer=lambda v:
      json.dumps(v).encode())

for _ in range(1_000):
    event = {
      "user_id":    random.randint(1, 500),
      "content_id": random.choice(CONTENT),
      "action":     random.choice(
                      ["play","pause","seek"]),
      "ts":         int(time.time())
    }
    producer.send(TOPIC, event)
producer.flush()
```

References

Seminal Paper

- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *Proc. NetDB Workshop at VLDB*.

Textbooks [SA], [TE], [TW]

- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 8 — Real-Time Streaming). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 10 — Lambda and Kappa Architectures). Prentice Hall.
- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 14 — Flume; Ch. 15 – Sqoop). O'Reilly.

Case Study Sources

- Arun, M. (2016). Kafka inside Keystone Pipeline. *Netflix Tech Blog*.
- Goodhope, K., et al. (2012). Building LinkedIn's Real-time Activity Data Pipeline. *IEEE Data*

Closing — Week 8, Part 2

Key Takeaways

- Kafka's insight — **treat the log as the database** — turned a simple append-only file into the backbone of modern event-driven architecture
- **Zero-copy + sequential I/O** are the two engineering decisions that explain Kafka's 10× throughput advantage over traditional MOM systems; they are also why Kafka runs on spinning disk at near-SSD throughput
- Netflix's pipeline shows that **Kafka's retention collapses the Lambda architecture's complexity**: when speed and batch layers both read from the same durable log, the dual-ingestion problem disappears
- **Schema governance and exactly-once**

Quote of the Week

"A log is perhaps the simplest possible storage abstraction. It is an append-only, totally ordered sequence of records ordered by time."

— Jay Kreps, *The Log* (2013)

Part 1 Preview — Week 9

Visualisation Principles, D3, Tableau & Grafana

- Tufte's data-ink ratio
- D3.js data joins and selections
- Tableau vs. Grafana use cases